Домашняя работа 3 - Cassandra

Домашняя работа посвящена Cassandra. Кейс - сервис аудита действий пользователя: каждое действие - вход, покупка или просмотр - пишется в таблицу, приложение читает ленту последних событий. В работе: модель данных и таблицы, вставка и чтение через PreparedStatement, замеры уровней консистентности ONE и QUORUM, сжатие SSTable, bloom-фильтр, TTL и компакция, nodetool repair. Стенд из трёх узлов и контрольные данные настроены; остаётся заполнить заготовку решения, прогнать сценарии и объяснить в README, как это работает и почему решения именно такие.

Короткая инструкция

1. Сложите `compose.yaml`, `lab.py`, `solution.py`, `init.cql`, `ops.sh` в одну папку - это папка проекта. Запустите Docker, откройте в ней терминал и поднимите кластер:

```
docker compose up -d
docker compose run --rm lab reset
```

Получится кластер из трёх узлов `c1`, `c2`, `c3` и keyspace `hw3`.

2. Посмотрите контрольные данные и эталоны:

```
docker compose run --rm lab fixtures
```

3. Заполняйте места с `TODO` в `solution.py`, после каждого блока запуская проверку:

Что сделано	Команда
Таблицы и модель данных	`docker compose run --rm lab data`
Вставка 10 000 записей и чтение	`docker compose run --rm lab bulk`
Замеры ONE и QUORUM	`docker compose run --rm lab consistency`
TTL и компакция	`docker compose run --rm lab compaction`

При ошибке исправьте решение и повторите команду.

4. Выполните ручные шаги:

```
bash ops.sh tablestats
bash ops.sh repair
bash ops.sh compact
docker compose run --rm lab metrics
```

5. Создайте README с объяснениями по вопросам задания; результаты сценариев уже в `results/`.

6. Итоговый прогон:

```
docker compose run --rm lab all
```

Запишите короткое видео с показом решения, добавьте исходники, README и результаты в репозиторий. Остановите стенд: `docker compose down`.

1. С чего начать

1. Подготовьте окружение по приложению А. В папке проекта уже есть `lab.py`, `init.cql`, `ops.sh` и заготовка `solution.py`.

2. Выполняйте разделы 2-8 по порядку, заполняя функции в `solution.py` и записывая объяснения в README.

2. Таблица и модель данных

Сервис пишет каждое действие пользователя в таблицу `user_actions`; действие - `login`, `purchase` или `view`. Чтение - лента последних событий пользователя. Контрольный пользователь - `user_id 1001`.

Таблица	Поля	Типы и ограничения
`user_actions`	`user_id`	int - partition key, идентификатор пользователя
`user_actions`	`event_time`	timestamp - clustering key, время события; порядок по убыванию
`user_actions`	`event_type`	text - `login`, `purchase` или `view`
`user_actions`	`payload`	text - детали события
`user_actions`	сжатие	LZ4
`user_actions_none`	те же поля	та же структура без сжатия; нужна для сравнения размеров в разделе 5

Что сделать

1. Заполните `create_tables` в `solution.py`: обе таблицы, первичный ключ (`user_id, event_time`), сортировка по убыванию `event_time`, сжатие LZ4 только для `user_actions`.

2. Проверьте схему сценарием `data` (primary key, порядок сортировки, сжатие).

3. Объясните в README: почему партиция по `user_id` и зачем обратный порядок кластеризации.

4. Объясните в README: что произойдёт при двух событиях одного пользователя с одинаковым `event_time`, почему для аудита это риск и как изменить схему, чтобы события не затирались.

3. Вставка и чтение

Запишите контрольный набор из 10 000 событий через PreparedStatement и прочитайте ленту последних событий.

Операция	Вход	Выход
`insert_events`	параметры заготовки	10 000 событий в `user_actions`, вставка через PreparedStatement
`read_last`	`user_id`	события пользователя, `event_time` по убыванию

Что сделать

1. Заполните `insert_events`: генерация 10 000 событий для контрольного пользователя, вставка через PreparedStatement.

2. Заполните `read_last`: чтение ленты с проверкой порядка - последние события первыми.

3. Прогоните сценарий `bulk` и заполните в README таблицу:

Что вставлено	Сколько	Время
События пользователя 1001 (`login`, `purchase`, `view`)	10 000	

Время - суммарное; среднее за одну запись вычислите, разделив на число записей. Числа печатаются в выводе сценария и сохраняются в `results/results.json`, раздел `artifacts.bulk` (секунды).

4. Объясните в README, что даёт PreparedStatement.

4. Консистентность

Сценарий `consistency` замеряет время записи и чтения при ONE и QUORUM на собственной нагрузке: 1000 записей, чтение 20 последних событий.

Что сделать

1. Прогоните `consistency` и заполните в README таблицу замеров:

Уровень	Запись, мс	Чтение, мс
ONE		
QUORUM		

Числа - медианы из 5 прогонов после прогрева; они в выводе сценария и в `results/results.json`, раздел `artifacts.consistency` (секунды; для таблицы пересчитать в миллисекунды), сырые прогоны - в поле `seconds_samples`. Консистентность задаётся на отдельных утверждениях (`statement.consistency_level`), а не на сессии.

2. Объясните формулу W + R > N. N = 3, replication factor в keyspace `hw3` тоже 3. Опишите, что меняется между ONE и QUORUM для записи и для чтения и какие гарантии даёт QUORUM.

3. Объясните, какой уровень выбрали бы для ленты событий пользователя и почему.

5. Сжатие

Сравните фактический размер `user_actions` (LZ4) и `user_actions_none` (без сжатия).

Что сделать

1. Выполните `bash ops.sh tablestats` после любого сценария, наполнившего таблицы (`bulk` или `consistency`). Перед снятием статистики скрипт делает `nodetool flush hw3`: размеры и bloom-фильтр считаются по SSTable, а не по memtable. Вывод сохраняется в `results/tablestats.txt` и `results/tablestats_plain.txt`.

2. Заполните таблицу сравнения по строкам `Space used (total)`, `SSTable count` и `SSTable Compression Ratio`:

Таблица	Space used (total)	SSTable count	SSTable Compression Ratio
`user_actions` (LZ4)			
`user_actions_none` (без сжатия)			

3. Объясните в README компромисс сжатия: что платится при записи и чтении и почему сжатие выгодно для этого кейса.

6. Bloom-фильтр

В `results/tablestats.txt` для таблицы `hw3.user_actions` найдите строки `Bloom filter false positives` и `Bloom filter false ratio`.

Что сделать

1. Зафиксируйте значения в README:

Показатель	Значение
Bloom filter false positives	
Bloom filter false ratio	

2. Объясните, что делает bloom-фильтр: почему при обращении к отсутствующему в таблице ключу disk lookup не нужен.

3. Объясните, что означает метрика и когда bloom-фильтр полезен, а когда не важен.

7. Repair

Что сделать

1. Выполните `bash ops.sh repair` и убедитесь, что появился файл `results/repair.txt`.

2. Объясните в README:

- что такое anti-entropy и зачем нужен `nodetool repair` - расхождения между репликами исправляются вне пути чтения;

- когда без repair не обойтись и что происходит при долгой работе без него: реплики расходятся, чтение может вернуть неактуальные данные, tombstones не подчищаются;

- чем repair отличается от read repair и hinted handoff: read repair чинит только ключи, затронутые чтением; hinted handoff компенсирует короткие простои узла хинтами.

8. TTL и компакция

Для журнала событий настраиваются TimeWindowCompactionStrategy и время жизни строк.

Что сделать

1. Заполните `configure_twcs_ttl` в `solution.py`: для `user_actions` - стратегия TWCS с окном в один день (`compaction_window_unit`, `compaction_window_size`) и `default_time_to_live` = 2592000 (30 дней).

2. Прогоните `compaction`: сценарий проверяет конфигурацию по `system_schema.tables`, вставляет 5 событий с TTL 5 секунд и убеждается, что после истечения строк нет.

3. В `cqlsh` (`docker compose run --rm lab shell`) включите `TRACING ON` и прочитайте партицию с истёкшими строками - в трейсе видна строка про tombstone.

4. Выполните `bash ops.sh compact`: принудительная компакция `user_actions` и сохранение истории в `results/compaction.txt`.

5. Объясните в README:

- почему TWCS подходит журналу событий, а STCS - нет;

- как TTL соотносится с gc_grace_seconds;

- почему TTL предпочтительнее массового DELETE.

9. Результаты и демонстрация

1. Результаты разделов 2-8 собраны в `results/`: сценарии - `results.json`, ручные шаги - `tablestats.txt`, `tablestats_plain.txt`, `repair.txt`, `compaction.txt`.

2. Добавьте в README объяснения из разделов 2-8 и команды воспроизведения. Заготовки таблиц с местами для чисел - в `results/README.md`.

3. В видео (ориентир 5-8 минут) покажите обязательные сценарии: `data`, `bulk`, замеры ONE и QUORUM, `tablestats` (сжатие и bloom), `compaction`, `repair` - и соответствующий код с фактическим выводом.

10. Оценивание

Максимум - **10 баллов**. Каждая часть оценивается целиком: ошибки одной части не обнуляют остальные. При снижении оценки указывается незачтённая часть и причина; дополнительных штрафов нет.

Критерий	Баллы	Из чего складывается оценка
Таблица и модель данных - раздел 2	**1,5**	Схема (PK, ORDER BY DESC) и сжатие LZ4 - 0,5; объяснение модели - 0,5; коллизия event_time - 0,5
Вставка и чтение - раздел 3	**1,5**	Вставка 10 000 через PreparedStatement - 0,5; проверка объёма и порядка - 0,5; объяснение PreparedStatement - 0,5
Консистентность - раздел 4	**2**	Замеры write/read при ONE и QUORUM - 1; объяснение W + R > N и выбор уровня - 1
Сжатие - раздел 5	**1**	Tablestats по обеим таблицам и таблица сравнения - 0,5; объяснение компромисса - 0,5
Bloom-фильтр - раздел 6	**0,5**	Метрика и объяснение - 0,5
Repair - раздел 7	**1**	Выполнен repair и сохранён вывод - 0,5; объяснения anti-entropy и последствий пропуска - 0,5
TTL и компакция - раздел 8	**1,5**	Конфигурация TWCS и TTL - 0,5; TTL-эксперимент и compactionhistory - 0,5; объяснения - 0,5
Результаты и демонстрация - раздел 9	**1**	Результаты в `results/` и README - 0,5; демо-видео - 0,5
Итого	**10**	

Приложение А. Подготовка и команды

Подготовка

1. Установите и запустите Docker с Compose v2. Для трёх узлов ориентир - от 2 ГБ памяти, доступной Docker: куча каждого узла ограничена 512 МБ. При первом запуске нужен интернет для скачивания образа. Cassandra и языковые среды на хосте не нужны.

2. Положите файлы комплекта в корень проекта: `compose.yaml`, `lab.py`, `solution.py`, `init.cql`, `ops.sh`. Откройте терминал в этой папке.

3. Поднимите окружение и сбросьте keyspace:

```
docker compose up -d
docker compose run --rm lab reset
```

4. Посмотрите данные и эталоны, затем заполняйте `solution.py`:

```
docker compose run --rm lab fixtures
```

Работа со стендом

Команда после `docker compose run --rm lab`	Действие
`data`	Проверить таблицы, схему (PK, ORDER BY DESC, LZ4) и фикстуры
`bulk`	Вставить 10 000 записей через PreparedStatement и проверить чтение
`consistency`	Замерить запись и чтение при ONE и QUORUM
`compaction`	Настроить TWCS+TTL, проверить TTL по `system_schema` и истечение строк
`metrics`	Проверить, что выполнены ручные шаги tablestats и repair
`all`	Последовательно выполнить data, compaction, bulk и consistency
`reset`	Сбросить keyspace `hw3` и подготовить начальное состояние; прежние результаты проверок очищаются
`fixtures`	Показать контрольные данные, параметры и эталоны
`shell`	Открыть интерактивный `cqlsh` для произвольных запросов
`help`	Показать справку

Команда на хосте	Действие
`bash ops.sh tablestats`	Выполнить `nodetool flush hw3`, затем `nodetool tablestats` по `user_actions` и `user_actions_none`; файлы `results/tablestats.txt` и `results/tablestats_plain.txt`
`bash ops.sh repair`	Выполнить `nodetool repair hw3`; файл `results/repair.txt`
`bash ops.sh compact`	Принудительная компакция `user_actions` и история компакций; файл `results/compaction.txt`

Например: `docker compose run --rm lab bulk`. Каждый сценарий готовит своё исходное состояние; не запускайте их одновременно и не меняйте контрольный набор. До заполнения соответствующих функций сообщение `TODO` ожидаемо. Для полного прогона используйте `all`.

Скрипт `ops.sh` вызывается через `bash ops.sh`, исполняемым его делать не нужно.

Результаты собраны в `results/`. Для остановки выполните `docker compose down`. Чтобы удалить и данные в Docker, выполните `docker compose down -v` и повторите подготовку; файлы результатов на хосте остаются.

Настройки

Параметр	Значение
Cassandra	Образ `cassandra:5.0.4`
Топология	Кластер из трёх узлов с данными: `c1`, `c2`, `c3`

Keyspace	hw3, replication factor 3
Таблицы	`user_actions` со сжатием LZ4; `user_actions_none` без сжатия
Компакция и TTL	TWCS окно 1 день, `default_time_to_live` 30 дней (раздел 8)
Доступ	Внутренняя Docker-сеть, без аутентификации и TLS; порты на хост не публикуются
Контактные точки	Имена сервисов `c1`, `c2`, `c3` во внутренней Docker-сети
Подключение	`docker compose exec c1 cqlsh`